

ISIS-1611: Inteligencia Artificial

Notas del Curso — Semestre 2026-19

Universidad de los Andes

Basado en: *Artificial Intelligence: A Modern Approach*

Russell, Norvig & Ming-Wei Chang (4^a ed.)

Carla González

9 de junio de 2026

Índice general

Parte I

Búsqueda

Capítulo 1

Semana 1: Introducción y Búsqueda Desinformada

1.1. Contexto e Historia de la IA

1.1.1. Impacto de la IA en el mundo real

La IA impacta múltiples sectores: economía, política, ley y derecho, trabajo, ciencia, salud y educación.

1.1.2. ¿Qué es la Inteligencia Artificial?

Existen cuatro enfoques para definir la IA:

Pensamiento humano Modelos cognitivos	Pensamiento racional Leyes del pensamiento (lógica)
Actuar humanamente Test de Turing	Actuar racionalmente Agentes racionales

Concepto clave

La IA moderna se enfoca en **agentes racionales**: entidades que perciben su entorno y actúan para maximizar su medida de desempeño esperada.

1.1.3. Historia de la IA

1.2. Agentes Inteligentes

1.2.1. ¿Qué es un Agente?

Definición 1.2.1 (Agente). Un agente es cualquier entidad que **percibe** su ambiente mediante **sensores** y **actúa** sobre él mediante **actuadores**.

La función del agente mapea percepciones a acciones: $f : P^* \rightarrow A$

1.2.2. Racionalidad

Un agente racional selecciona la acción que maximiza su **medida de desempeño** esperada, dado su conocimiento actual.

Depende de:

1. La medida de desempeño

2. El conocimiento previo del entorno
3. Las acciones disponibles
4. La secuencia de percepciones hasta el momento

1.2.3. Descripción PEAS

Para caracterizar el ambiente de una tarea se usa PEAS:

- **Performance** (Medida de desempeño)
- **Environment** (Entorno)
- **Actuators** (Actuadores)
- **Sensors** (Sensores)

1.2.4. Naturaleza de los Ambientes

Dimensión	Valores posibles
Observabilidad	Completamente observable / Parcialmente observable
Agentes	Un solo agente / Multiagente
Determinismo	Determinista / Estocástico
Episodicidad	Episódico / Secuencial
Dinamismo	Estático / Dinámico
Tiempo	Discreto / Continuo
Conocimiento	Conocido / Desconocido

1.2.5. Tipos de Agentes

1. **Agente de reflejo simple**: actúa según la percepción actual (reglas condición-acción).
2. **Agente de reflejo basado en modelos**: mantiene un estado interno del mundo.
3. **Agente basado en objetivos**: considera metas para elegir acciones.
4. **Agente basado en utilidad**: maximiza una función de utilidad.
5. **Agente que aprende**: mejora su desempeño con la experiencia.

1.3. Búsqueda Desinformada I: Formulación del Problema

1.3.1. Formulación de un Problema de Búsqueda

Definición 1.3.1 (Problema de búsqueda). Un problema de búsqueda se define con cinco componentes:

1. **Estado inicial**: s_0
2. **Modelo de transición**: $\text{RESULT}(s, a) \rightarrow s'$
3. **Función de costo de acción**: $c(s, a, s')$
4. **Test de meta**: $\text{IS-GOAL}(s) \rightarrow \{T, F\}$
5. **Espacio de estados**: grafo de todos los estados alcanzables

La **solución** es una secuencia de acciones que lleva del estado inicial a un estado meta. Una **solución óptima** minimiza el costo total del camino.

1.3.2. Nodos vs. Estados

Un **nodo** en el árbol de búsqueda contiene:

- El estado que representa
- El nodo padre
- La acción que generó el nodo
- El costo del camino $g(n)$
- La profundidad

1.3.3. Medidas de Desempeño de Algoritmos

Criterio	Descripción
Compleitud	¿Encuentra solución si existe?
Optimalidad	¿Encuentra la solución de menor costo?
Complejidad en tiempo	Nodos generados/expandidos
Complejidad en espacio	Nodos en memoria simultáneamente

Los parámetros usuales son b (factor de ramificación), d (profundidad de la solución más superficial) y m (profundidad máxima del árbol).

1.4. Búsqueda Desinformada II: Algoritmos

1.4.1. Breadth-First Search (BFS)

- Expande el nodo más **superficial** primero (FIFO).
- **Completo**: sí (si b finito).
- **Óptimo**: sí, si todos los costos de paso son iguales.
- Tiempo: $\mathcal{O}(b^d)$ Espacio: $\mathcal{O}(b^d)$

1.4.2. Depth-First Search (DFS)

- Expande el nodo más **profundo** primero (LIFO).
- **Completo**: no en árbol infinito; sí en grafo finito.
- **Óptimo**: no.
- Tiempo: $\mathcal{O}(b^m)$ Espacio: $\mathcal{O}(bm)$

1.4.3. Depth-Limited Search y Iterative Deepening (IDDFS)

IDDFS ejecuta DFS con límite de profundidad creciente $l = 0, 1, 2, \dots$

- Combina las ventajas de BFS (completo, óptimo si costos iguales) con las de DFS (espacio lineal).
- Tiempo: $\mathcal{O}(b^d)$ Espacio: $\mathcal{O}(bd)$

1.4.4. Uniform-Cost Search (UCS)

- Expande el nodo con menor **costo de camino** $g(n)$ (cola de prioridad).
- Equivale a BFS cuando todos los costos son iguales.
- **Completo**: sí (si costos $> \epsilon > 0$).
- **Óptimo**: sí.
- Tiempo y espacio: $\mathcal{O}(b^{1+\lceil C^*/\epsilon \rceil})$, donde C^* es el costo óptimo.

1.4.5. Comparativa Búsqueda Desinformada

Algoritmo	Completo	Óptimo	Tiempo	Espacio
BFS	Sí	Sí*	b^d	b^d
DFS	No	No	b^m	bm
IDDFS	Sí	Sí*	b^d	bd
UCS	Sí	Sí	$b^{C^*/\epsilon}$	$b^{C^*/\epsilon}$

*Óptimo si costos de paso son iguales

1.5. Búsqueda Informada: Heurísticas

1.5.1. Función Heurística

Definición 1.5.1 (Heurística). $h(n)$ es una función que estima el costo desde el nodo n hasta la meta más cercana. Debe ser **admisible**: nunca sobreestima el costo real.

Importante

Una heurística es **admisible** si $h(n) \leq h^*(n)$ para todo n , donde $h^*(n)$ es el costo real al llegar a la meta.

1.5.2. Greedy Best-First Search

Expande el nodo que parece más cercano a la meta según $h(n)$.

- Función de evaluación: $f(n) = h(n)$
- **No es óptimo** ni completo en general.
- Rápido en la práctica, puede quedar atrapado en mínimos locales.

1.5.3. A*

Combina el costo real del camino y la heurística:

$$f(n) = g(n) + h(n)$$

- $g(n)$: costo del camino desde inicio hasta n
- $h(n)$: estimación del costo desde n hasta la meta
- **Completo** y **óptimo** si h es admisible (búsqueda en árbol) o consistente (búsqueda en grafo).

Definición 1.5.2 (Consistencia / Monotonía). h es consistente si para todo nodo n y sucesor n' :

$$h(n) \leq c(n, a, n') + h(n')$$

La consistencia implica admisibilidad.

Capítulo 2

Semana 2: Búsqueda Informada y Ambientes Complejos

Eventos esta semana

- Publicación **Taller 1**
- **Examen 1** (al final de la semana)

2.1. Búsqueda Informada Avanzada

2.1.1. Búsqueda con Memoria Acotada

Iterative Deepening A* (IDA*)

Versión de A* que usa IDDFS con el valor de corte $f(n) = g(n) + h(n)$.

- Espacio: $\mathcal{O}(bd)$
- Completo y óptimo (con h admisible)

Recursive Best-First Search (RBFS)

Mantiene el mejor valor alternativo f en cada nivel de recursión.

- Espacio: $\mathcal{O}(bd)$
- Puede reexpandir nodos (regenera sub-árboles olvidados)

SMA* (Simplified Memory-Bounded A*)

Usa toda la memoria disponible; descarta el nodo con mayor f cuando la memoria está llena.

2.1.2. Heurísticas: Dominancia y Creación

Definición 2.1.1 (Dominancia). h_2 domina a h_1 si $h_2(n) \geq h_1(n)$ para todo n y ambas son admisibles. Una heurística dominante expande menos nodos.

Formas de construir heurísticas admisibles:

1. **Relajación del problema:** eliminar restricciones del problema original.
2. **Bases de datos de patrones:** precomputar costos exactos de subproblemas.
3. **Máximo de heurísticas:** $h(n) = \max(h_1(n), h_2(n))$

2.2. Búsqueda en Ambientes Complejos

2.2.1. Búsqueda Local

A diferencia de la búsqueda sistemática, la búsqueda local opera sobre un **estado actual** sin mantener el camino recorrido. Útil cuando el objetivo es el estado mismo (no el camino).

2.2.2. Hill Climbing (Ascenso de Colina)

Algorithm 1 Hill-Climbing

```

1: current ← estado inicial
2: loop
3:   neighbor ← mejor sucesor de current
4:   if  $f(\text{neighbor}) \leq f(\text{current})$  then return current
5:   end if
6:   current ← neighbor
7: end loop

```

Problemas:

- Máximos locales (no globales)
- Mesetas (*plateaus*) y crestas

Variantes: Hill climbing estocástico, reinicio aleatorio, *simulated annealing*.

2.2.3. Simulated Annealing

Permite movimientos malos con probabilidad $e^{\Delta E/T}$ donde T decrece con el tiempo. Garantiza convergencia al óptimo global si T baja suficientemente despacio.

2.2.4. Algoritmos Genéticos

Inspirados en la evolución biológica. Operan sobre una **población** de individuos (estados codificados como cadenas).

Operadores:

1. **Selección:** individuos con mayor aptitud (*fitness*) tienen más probabilidad de reproducirse.
2. **Cruzamiento** (*crossover*): combina dos individuos para producir descendencia.
3. **Mutación:** cambia aleatoriamente genes con baja probabilidad.

2.3. Ejercicios de Búsqueda

Nota

Esta clase es de ejercicios y repaso para el **Examen 1**. Ver la *Guía de Ejercicios: Búsqueda Desinformada e Informada*.

2.3.1. Guía de Ejercicios — Resumen de Bloques

Bloque 1: Agentes Inteligentes

Caracterizar escenarios con PEAS, identificar tipo de agente y entorno.

Bloque 2: Formulación de Problemas

Definir estado inicial, modelo de transición, costo y test de meta para problemas concretos.

Bloque 3: Búsqueda No Informada

Aplicar BFS, DFS y UCS sobre grafos; analizar completitud y optimalidad.

Bloque 4: Búsqueda Informada y Heurísticas

Aplicar Greedy, A*, RBFS, SMA*; verificar admisibilidad y consistencia de heurísticas.

Ejercicio 2.3.1. Dado el grafo del Bloque 3 (Guía), aplica BFS, DFS y UCS. Indica el orden de expansión y el camino solución encontrado.

Solución:

Parte II

Razonamiento bajo Incertidumbre

Capítulo 3

Semana 3: Búsqueda Adversaria y Razonamiento bajo Incertidumbre

Eventos esta semana

- Entrega Taller 1

3.1. Búsqueda Adversaria y Juegos I

3.1.1. Juegos como Problemas de Búsqueda

Un **juego** de dos jugadores, suma cero, de información perfecta se define como:

- S_0 : estado inicial
- $\text{PLAYER}(s)$: qué jugador mueve en estado s
- $\text{ACTIONS}(s)$: movimientos legales
- $\text{RESULT}(s, a)$: estado resultante
- $\text{IS-TERMINAL}(s)$: test de fin de juego
- $\text{UTILITY}(s, p)$: valor numérico del estado terminal para el jugador p

3.1.2. Minimax

MAX maximiza, MIN minimiza la utilidad.

$$\text{MINIMAX}(s) = \begin{cases} \text{UTILITY}(s, \text{MAX}) & \text{si IS-TERMINAL}(s) \\ \max_a \text{MINIMAX}(\text{RESULT}(s, a)) & \text{si PLAYER}(s) = \text{MAX} \\ \min_a \text{MINIMAX}(\text{RESULT}(s, a)) & \text{si PLAYER}(s) = \text{MIN} \end{cases}$$

- Completo (en árbol finito) y óptimo contra un oponente óptimo.
- Tiempo: $\mathcal{O}(b^m)$ Espacio: $\mathcal{O}(bm)$

3.2. Búsqueda Adversaria y Juegos II

3.2.1. Poda Alfa-Beta (α - β)

Elimina ramas del árbol que no pueden afectar la decisión final.

- α : mejor valor que MAX puede garantizar (lo que MAX ya tiene)
- β : mejor valor que MIN puede garantizar (lo que MIN ya tiene)
- Poda: si $v \geq \beta$ en nodo MAX $\Rightarrow$ cortar; si $v \leq \alpha$ en nodo MIN $\Rightarrow$ cortar.

Con orden perfecto de movimientos: $\mathcal{O}(b^{m/2})$ (duplica la profundidad manejable).

3.2.2. Juegos con Incertidumbre: Expectiminimax

Para juegos con elementos aleatorios (dados) se agregan **nodos azar**:

$$\text{EXPECTIMINIMAX}(s) = \sum_r P(r) \cdot \text{EXPECTIMINIMAX}(\text{RESULT}(s, r))$$

3.2.3. Funciones de Evaluación

En juegos reales no se puede explorar hasta el final. Se usa una **función de evaluación heurística** $\text{EVAL}(s)$ que estima la utilidad desde un estado no terminal.

Características de una buena función de evaluación:

- Debe concordar con la utilidad real en estados terminales.
- Rápida de calcular.
- Debe reflejar las probabilidades reales de ganar.

3.2.4. Monte Carlo Tree Search (MCTS)

Estrategia basada en simulaciones aleatorias (*rollouts*). Cuatro fases: selección, expansión, simulación, retropropagación.

3.3. Razonamiento bajo Incertidumbre I

3.3.1. Probabilidad como Base del Razonamiento

Cuando el entorno es **parcialmente observable** o **estocástico**, se usa probabilidad para representar la incertidumbre.

Concepto clave

La **probabilidad a priori** (o incondicional) $P(A)$ representa el grado de creencia sin ninguna evidencia adicional.

La **probabilidad condicional** $P(A \mid B) = \frac{P(A \wedge B)}{P(B)}$ representa el grado de creencia dado que B es verdadero.

3.3.2. Regla de Bayes

$$P(A \mid B) = \frac{P(B \mid A) \cdot P(A)}{P(B)}$$

- $P(A)$: probabilidad a priori de A (hipótesis)
- $P(B \mid A)$: verosimilitud de la evidencia B dado A

- $P(B)$: probabilidad marginal de B (normalizador)
- $P(A | B)$: probabilidad a posteriori

3.3.3. Distribuciones de Probabilidad

Para variables aleatorias discretas, la **distribución conjunta completa** especifica $P(X_1, X_2, \dots, X_n)$ para todos los valores posibles. De ella se puede derivar cualquier probabilidad por marginalización y condicionamiento.

Marginalización: $P(X) = \sum_y P(X, Y = y)$

Independencia: $X \perp Y \Leftrightarrow P(X, Y) = P(X) \cdot P(Y)$

Independencia condicional: $X \perp Y | Z \Leftrightarrow P(X | Y, Z) = P(X | Z)$

3.4. Razonamiento bajo Incertidumbre II

3.4.1. Inferencia por Enumeración

Para cualquier consulta $P(X | \mathbf{e})$:

$$P(X | \mathbf{e}) = \alpha \sum_{\mathbf{y}} P(X, \mathbf{e}, \mathbf{y})$$

donde $\mathbf{y}$ son las variables ocultas y α es el normalizador.

3.4.2. Variables y Modelos

- **Variable aleatoria:** puede ser discreta o continua.
- **Distribución de probabilidad:** asigna probabilidades a todos los valores.
- **Tabla de probabilidad condicional (CPT):** especifica $P(X | \text{Parents}(X))$.

Capítulo 4

Semana 4: Redes Bayesianas, MDPs y CSP

Eventos esta semana

- Publicación **Taller 2**

4.1. Redes Bayesianas

Definición 4.1.1 (Red Bayesiana). Una red bayesiana es un grafo acíclico dirigido (DAG) donde:

- Cada nodo representa una variable aleatoria.
- Los arcos representan dependencias directas.
- Cada nodo X_i tiene una CPT: $P(X_i \mid \text{Parents}(X_i))$.

4.1.1. Semántica

La distribución conjunta se factoriza como:

$$P(X_1, \dots, X_n) = \prod_{i=1}^n P(X_i \mid \text{Parents}(X_i))$$

4.1.2. Inferencia Exacta

- **Enumeración**: suma sobre todas las variables ocultas. Exponencial en el peor caso.
- **Eliminación de variables**: orden de eliminación importa para la eficiencia.

4.1.3. Inferencia Aproximada

- **Muestreo directo**: muestrea según el orden topológico de la red.
- **Muestreo por rechazo**: descarta muestras inconsistentes con la evidencia.
- **Ponderación por verosimilitud**: fija la evidencia, pondera las muestras.
- **MCMC / Gibbs Sampling**: muestrea cada variable condicionada al resto.

4.2. Procesos de Decisión de Markov (MDPs)

Definición 4.2.1 (MDP). Un MDP se define por la tupla (S, A, P, R, γ) :

- S : conjunto de estados
- A : conjunto de acciones
- $P(s' | s, a)$: modelo de transición (probabilístico)
- $R(s, a, s')$: función de recompensa
- $\gamma \in [0, 1]$: factor de descuento

4.2.1. Propiedad de Markov

$$P(s_{t+1} | s_t, a_t) = P(s_{t+1} | s_0, a_0, \dots, s_t, a_t)$$

El futuro solo depende del estado presente, no del historial.

4.2.2. Política y Utilidad

- **Política** $\pi(s)$: función que asigna una acción a cada estado.
- **Utilidad** (retorno descontado): $U = \sum_{t=0}^{\infty} \gamma^t R_t$
- **Política óptima** π^* : maximiza la utilidad esperada.

4.2.3. Ecuación de Bellman

$$U^*(s) = \max_a \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma U^*(s')]$$

4.2.4. Algoritmos de Solución

- **Value Iteration**: actualiza $U(s)$ iterativamente hasta convergencia.
- **Policy Iteration**: alterna entre evaluación de política y mejora de política.

4.3. Ejercicios: Razonamiento bajo Incertidumbre

Nota

Repaso de probabilidad, redes bayesianas y MDPs.

Ejercicio 4.3.1. Dada la red bayesiana del ejemplo (Burglar-Earthquake-Alarm), calcula $P(\text{Burglary} | \text{JohnCalls} = T, \text{MaryCalls} = T)$ usando enumeración.

Solución:

4.4. Problemas de Satisfacción de Restricciones (CSP)

Definición 4.4.1 (CSP). Un CSP se define por:

- $X = \{X_1, \dots, X_n\}$: variables
- $D = \{D_1, \dots, D_n\}$: dominios de cada variable
- C : conjunto de restricciones sobre subconjuntos de variables

La **solución** es una asignación completa y consistente.

4.4.1. Tipos de Restricciones

- **Unarias**: $X_i \in \{v_1, v_2\}$
- **Binarias**: $X_i \neq X_j$
- **Globales**: involucran más de 2 variables (ej. `AllDiff`)

4.4.2. Búsqueda con Retroceso (Backtracking)

Asigna valores a variables una por una; retrocede cuando se viola una restricción.

Heurísticas para mejorar el backtracking:

1. **MRV** (Minimum Remaining Values): elegir la variable con menos valores posibles.
2. **Degree heuristic**: variable con más restricciones sobre variables no asignadas.
3. **Least Constraining Value**: valor que deja más opciones para las demás variables.

Capítulo 5

Semana 5: CSP Avanzado y Agentes Lógicos

Eventos esta semana

- Entrega Taller 2
- Examen 2

5.1. CSP Avanzado: Propagación de Restricciones

5.1.1. Consistencia de Arco (AC-3)

Una variable X_i es **arco-consistente** con X_j si para cada valor en D_i existe algún valor en D_j que satisface la restricción binaria.

AC-3 reduce dominios iterativamente hasta que no quedan inconsistencias o algún dominio queda vacío (sin solución).

- Complejidad: $\mathcal{O}(cd^3)$ donde c = número de restricciones, d = tamaño del dominio.

5.1.2. Forward Checking

Al asignar un valor a X_i , eliminar de los dominios de las variables vecinas no asignadas todos los valores inconsistentes.

5.1.3. Estructura del Grafo de Restricciones

Si el grafo de restricciones es un **árbol**, el CSP puede resolverse en $\mathcal{O}(nd^2)$ sin backtracking.

Cutset conditioning: encontrar un conjunto de variables cuya eliminación convierte el grafo en árbol.

5.1.4. Búsqueda Local para CSP

Min-conflicts: en cada paso, reparar la variable con más conflictos eligiendo el valor que minimiza conflictos.

- Muy eficiente en la práctica para CSPs grandes.
- No garantiza completitud.

5.2. Ejercicios: Problemas de Satisfacción de Restricciones

Nota

Preparación para el **Examen 2**. Aplicar backtracking con MRV, AC-3 y min-conflicts.

Ejercicio 5.2.1. Aplica backtracking con MRV y AC-3 al problema de coloreado de mapa de Australia con 3 colores.

Solución:

5.3. Agentes Lógicos: Introducción

5.3.1. Agentes Basados en Conocimiento

Un agente basado en conocimiento mantiene una **base de conocimiento** (KB) y utiliza **inferencia** para decidir qué acción tomar.

$$KB \vdash \alpha \quad (\text{KB implica } \alpha)$$

5.3.2. Lógica Proposicional

Conectivos lógicos: $\neg, \wedge, \vee, \Rightarrow, \Leftrightarrow$

- Una **oración** es *válida* (tautología) si es verdadera en todos los modelos.
- Una oración es *satisfacible* si es verdadera en al menos un modelo.
- **Consecuencia lógica:** $KB \models \alpha$ si en todo modelo donde KB es verdadera, α también lo es.

5.3.3. Formas Normales

- **CNF** (Forma Normal Conjuntiva): conjunción de cláusulas (disyunciones de literales).
- **DNF** (Forma Normal Disyuntiva): disyunción de conjunciones.

Parte III

Lógica y Representación del Conocimiento

Capítulo 6

Semana 6: Lógica y Representación del Conocimiento

6.1. Agentes Lógicos II: Inferencia Proposicional

6.1.1. Reglas de Inferencia

Regla	Forma
Modus Ponens	$\alpha \Rightarrow \beta, \alpha \vdash \beta$
Modus Tollens	$\alpha \Rightarrow \beta, \neg\beta \vdash \neg\alpha$
Resolución	$(\ell_1 \vee \dots \vee \ell_k), (\neg\ell_1 \vee m_1 \vee \dots) \vdash (m_1 \vee \dots)$

6.1.2. Resolución

Algoritmo de inferencia completo para lógica proposicional en CNF:

1. Convertir $KB \wedge \neg\alpha$ a CNF.
2. Aplicar resolución hasta obtener la cláusula vacía (contradicción) o no poder seguir.
3. Si se obtiene contradicción $\Rightarrow KB \models \alpha$.

6.1.3. DPLL y WalkSAT

- **DPLL**: backtracking completo sobre asignaciones de verdad, con propagación de cláusulas unitarias.
- **WalkSAT**: búsqueda local incompleta, eficaz en la práctica para instancias satisfacibles.

6.2. Lógica de Primer Orden (LPO)

6.2.1. Sintaxis

Elementos de LPO:

- **Constantes**: *Juan, UNIANDES, ...*
- **Variables**: $x, y, z, \dots$
- **Predicados**: $Estudiante(x), Mayor(x, y)$
- **Funciones**: $Padre(x), Suma(x, y)$

- **Cuantificadores:** $\forall$ (universal), $\exists$ (existencial)
- **Conectivos:** $\neg, \wedge, \vee, \Rightarrow, \Leftrightarrow$

6.2.2. Semántica

Un **modelo** en LPO especifica:

- El dominio de objetos
- La interpretación de constantes, predicados y funciones

6.2.3. Cuantificadores

$$\forall x P(x) \equiv \neg \exists x \neg P(x)$$

$$\exists x P(x) \equiv \neg \forall x \neg P(x)$$

Importante

$\forall x (P(x) \Rightarrow Q(x))$ es diferente de $\forall x (P(x) \wedge Q(x))$. El cuantificador universal normalmente usa $\Rightarrow$; el existencial usa $\wedge$.

6.3. LPO e Inferencia I

6.3.1. Bases de Conocimiento en LPO

Las bases de conocimiento en LPO pueden expresar relaciones complejas de forma compacta, a diferencia de la lógica proposicional.

6.3.2. Unificación

Para aplicar reglas de inferencia en LPO se necesita **unificar** expresiones:

$$\text{UNIFY}(\alpha, \beta) = \theta \text{ tal que } \text{SUBST}(\theta, \alpha) = \text{SUBST}(\theta, \beta)$$

Algoritmo: recorre la estructura de ambas expresiones, construyendo sustituciones. Devuelve *fallo* si no hay unificador.

6.3.3. Inferencia con Cuantificadores

- **Instanciación universal:** $\forall x \alpha(x) \vdash \alpha(c)$ para cualquier constante c .
- **Instanciación existencial:** $\exists x \alpha(x) \vdash \alpha(k)$ donde k es una nueva constante de Skolem.
- **Skolemización:** eliminar cuantificadores existenciales introduciendo constantes/funciones de Skolem.

6.3.4. Forma Normal de Clausura (CNF en LPO)

Pasos para convertir a CNF:

1. Eliminar bicondicionales e implicaciones.
2. Mover $\neg$ hacia adentro (De Morgan, reglas de cuantificadores).

3. Estandarizar variables.
4. Skolemizar.
5. Eliminar cuantificadores universales.
6. Distribuir $\vee$ sobre $\wedge$.

6.4. Inferencia en LPO II

6.4.1. Encadenamiento hacia adelante (Forward Chaining)

Aplica reglas de la forma $P_1 \wedge \dots \wedge P_n \Rightarrow Q$ para derivar nuevos hechos hasta alcanzar la meta.

- Completo para bases de conocimiento de **cláusulas de Horn**.
- Dirigido por los datos.

6.4.2. Encadenamiento hacia atrás (Backward Chaining)

Parte de la meta y trabaja hacia atrás buscando hechos o submetas.

- Completo para cláusulas de Horn.
- Base de Prolog.
- Puede ciclar si no se controla.

6.4.3. Resolución en LPO

Generaliza la resolución proposicional usando unificación:

$$\frac{\ell_1 \vee \dots \vee \ell_k \quad \neg m_1 \vee \dots \vee \neg m_j}{\text{SUBST}(\theta, \ell_1 \vee \dots \vee m_j)}$$

donde $\theta = \text{UNIFY}(\ell_i, m_k)$.

- **Refutación por resolución:** completa para LPO (Teorema de Completitud de Gödel).

Parte IV

Planeación

Capítulo 7

Semana 7: Inferencia en LPO y Planeación

Eventos esta semana

- Publicación Taller 3

7.1. Inferencia en LPO: Cierre

Nota

Repaso de resolución en LPO con ejemplos. Transición hacia planeación.

7.2. Planeación: Definición y Algoritmos Generales

7.2.1. ¿Qué es la Planeación?

Un agente planificador construye una **secuencia de acciones** para lograr un objetivo en un entorno modelado.

Definición 7.2.1 (Problema de Planeación STRIPS/PDDL).
■ **Estado:** conjunción de fluentes (literales positivos/negativos).

- **Acción:** definida por precondiciones y efectos.
- **Meta:** conjunción de fluentes que deben ser verdaderos.

7.2.2. Representación PDDL

Listing 7.1: Ejemplo de acción en PDDL

```
1 (:action Fly
2   :parameters (?p - Plane ?from ?to - Airport)
3   :precondition (and (At ?p ?from) (Plane ?p)
4                     (Airport ?from) (Airport ?to))
5   :effect (and (At ?p ?to) (not (At ?p ?from))))
```

7.2.3. Algoritmos de Búsqueda para Planeación

- **Búsqueda hacia adelante** (*progression*): empieza en el estado inicial y aplica acciones.
- **Búsqueda hacia atrás** (*regression*): empieza desde la meta y aplica acciones inversas.
- **SATPLAN**: codifica el problema de planeación como SAT y usa un solucionador SAT.

7.3. Planeación: Heurísticas y Planeación Jerárquica

7.3.1. Heurísticas para Planeación

Problema Relajado (ignore preconditions / ignore delete lists)

- **Ignore preconditions**: todas las acciones son aplicables. Heurística: número mínimo de acciones para satisfacer la meta.
- **Ignore delete lists**: no se eliminan fuentes. Problema de planeación relajado resoluble en tiempo polinomial.

Planning Graph y Heurística FF

Un **grafo de planeación** alterna capas de literales y acciones:

- Capa de literales S_i : literales alcanzables en i pasos.
- Capa de acciones A_i : acciones aplicables en el paso i .

La heurística FF usa el plan del grafo relajado como estimador.

7.3.2. Planeación Jerárquica (HTN)

Hierarchical Task Network (HTN): descompone tareas abstractas en subtareas más concretas.

- **Tareas primitivas**: acciones directamente ejecutables.
- **Tareas abstractas**: se descomponen mediante métodos.
- **Ventaja**: permite incorporar conocimiento de dominio para guiar la búsqueda.

7.4. Planeación en Ambientes No Deterministas

7.4.1. Ambientes No Deterministas y Condicionales

Cuando las acciones tienen efectos inciertos, se necesitan **planes condicionales** que incluyan ramificaciones según el resultado de las acciones.

- **Plan de búsqueda AND-OR**: los nodos AND representan contingencias del entorno; los nodos OR representan elecciones del agente.
- Un plan condicional es una solución si cubre todas las ramas.

7.4.2. Planeación Parcialmente Observable

Con observabilidad parcial el agente mantiene un **estado de creencia** (belief state) y ejecuta acciones de recopilación de información.

7.4.3. Planeación Continua y de Ejecución

- **Monitoreo de ejecución:** detectar discrepancias entre el plan y la realidad.
- **Replaneación:** adaptar el plan ante fallos o nueva información.
- **Agentes reactivos-deliberativos:** combinan planeación a largo plazo con respuesta reactiva.

Parte V

IA Moderna, Aplicaciones y Ética

Capítulo 8

Semana 8: Aplicaciones, Ética y Repaso Final

Eventos esta semana

- Entrega Taller 3
- Examen Final

8.1. Aplicaciones en IA

8.1.1. Machine Learning

Paradigma donde los sistemas **aprenden de datos** en lugar de ser programados explícitamente.

- **Aprendizaje supervisado:** clasificación, regresión.
- **Aprendizaje no supervisado:** clustering, reducción de dimensionalidad.
- **Aprendizaje por refuerzo (RL):** aprender mediante interacción con el entorno y recompensas.

8.1.2. Deep Learning

Redes neuronales profundas con múltiples capas de abstracción.

- CNN (Convolutional Neural Networks): visión computacional.
- RNN / Transformers: procesamiento de lenguaje natural (NLP).
- LLMs: modelos de lenguaje de gran escala (ej. GPT, Claude).

8.1.3. Áreas de Aplicación

- **NLP:** traducción automática, chatbots, análisis de sentimientos.
- **Visión computacional:** detección de objetos, reconocimiento facial, diagnóstico médico por imágenes.
- **Robótica:** percepción, planeación de movimiento, manipulación.

8.2. Ética en IA y Aplicaciones Futuras

8.2.1. Riesgos y Desafíos Éticos

- **Sesgo algorítmico:** sistemas entrenados con datos sesgados perpetúan o amplifican discriminación.
- **Privacidad:** recolección masiva de datos personales.
- **Seguridad:** ataques adversariales, manipulación de sistemas autónomos.
- **Desplazamiento laboral:** automatización de tareas cognitivas y manuales.
- **Concentración de poder:** dominio de pocas empresas sobre infraestructura de IA.

8.2.2. Marcos Regulatorios

- **AI Act** (Unión Europea): regulación por niveles de riesgo.
- Lineamientos de uso de IA generativa en contextos académicos.

8.2.3. Niveles de Uso de IA Generativa (Uniandes)

Nivel	Uso	Descripción
1	Sin IA generativa	No debe usarse en ningún aspecto
2	Generar/estructurar ideas	No incluir material generado en el trabajo
3	Editar y refinar	Explicar uso en nota al pie o anexo
4	Completar tareas (revisión humana)	Distinguir material IA; incluir transcripción de prompts
5	Uso pleno	Sin restricciones de distinción

8.2.4. Futuro de la IA

8.3. Repaso para Examen Final

Nota

Repaso de todos los temas del curso en preparación para el **Examen Final**. El examen cubre: búsqueda, incertidumbre, CSP, lógica y planeación.

8.3.1. Resumen de Temas

1. **Agentes inteligentes:** PEAS, tipos de agente, naturaleza del entorno.
2. **Búsqueda no informada:** BFS, DFS, IDDFS, UCS.
3. **Búsqueda informada:** A*, IDA*, RBFS; admisibilidad y consistencia.
4. **Búsqueda local:** Hill climbing, simulated annealing, algoritmos genéticos.
5. **Búsqueda adversaria:** Minimax, poda α - β , MCTS.
6. **Incertidumbre:** probabilidad, Bayes, redes bayesianas, inferencia, MDPs.
7. **CSP:** backtracking, AC-3, MRV, min-conflicts.
8. **Lógica:** proposicional, LPO, resolución, encadenamiento.
9. **Planeación:** PDDL, búsqueda en espacio de planes, heurísticas, HTN.